

108. Convert Sorted Array to Height-Balanced BST

PLATFORM

Platform: LeetCode

DIFFICULTY

EASY

DATE

Solved: August 3, 2021

Problem Summary

Given an integer array sorted in ascending order, convert it to a height-balanced BST where depth of left and right subtrees of every node differs by at most 1.

Algo

1. build(nums, left, right) — if left > right return null.
2. mid = (left + right) / 2.
3. Create node = new TreeNode(nums[mid]).
4. node.left = build(nums, left, mid-1).
5. node.right = build(nums, mid+1, right).
6. Return node.

Time Complexity & Space Complexity

$O(n)$ — every element becomes a node exactly once.

$O(\log n)$ — recursion stack depth equals tree height (balanced).

Approach

Recursively pick the middle element of the current subarray as root. Left half builds left subtree, right half builds right subtree. This guarantees balance since both halves are always equal (or differ by 1).

Key Insights

- Middle element as root → equal elements on both sides → balanced height.
- Same divide and conquer idea as **binary search** — split at mid every time.
- left > right base case handles empty subarray — returns null naturally.
- mid+1 for right half — never include mid again or you get infinite recursion.

Observations

- Multiple valid answers exist — any mid choice (left-mid or right-mid) produces a valid balanced BST.
- Same pattern used in **merge sort** and **binary search** — recognizing divide and conquer saves time.
- Sorted linked list to BST (LC 109) is the harder version — can't do random access, needs slow/fast pointer for mid.

Approaches

- **Brute Force:** Insert elements one by one into BST → $O(n \log n)$ but NOT guaranteed balanced.
- **Optimal $O(n)$:** Divide and conquer — always pick middle element as root, recurse on left and right halves.

Problem List

108. Convert Sorted Array to Binary Search Tree

Solved

Easy

Topics

Companies

Given an integer array `nums` where the elements are sorted in **ascending order**, convert it to a **height-balanced** binary search tree.

Example 1:

```
graph TD; 0 --- -3; 0 --- 9; -3 --- -10; 9 --- 5
```

Input: `nums = [-10,-3,0,5,9]`
Output: `[0,-3,9,-10,null,5]`
Explanation: `[0,-10,5,null,-3,null,9]` is also accepted:

```
graph TD; 0 --- -10; 0 --- 5; -10 --- -3; 5 --- 9
```

Example 2:

Accepted

31 / 31 testcases passed

Sahil Khan submitted at Aug 03, 2026 11:10

₹583.25/mo

Save over 58%

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one...

Runtime

0 ms | Beats 100.00%

Memory

45.03 MB | Beats 45.35%

Submission	Runtime (ms)
User	0
Other 1	1
Other 2	2
Other 3	3
Other 4	4

Code

Java

```
11 * this.left = left;
12 * this.right = right;
13 * }
14 * }
15 */
16 class Solution {
17     public TreeNode sortedArrayToBST(int[] nums) {
18         return build(nums, 0, nums.length-1);
19     }
20
21     TreeNode build(int[] nums, int left, int right){
22         if(left > right) return null;
23
24         int mid = (left+right)/2;
25
26         TreeNode node = new TreeNode(nums[mid]);
27
28         node.left = build(nums, left, mid-1);
29         node.right = build(nums, mid+1, right);
30
31         return node;
32     }
33 }
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

nums = [-10,-3,0,5,9]

Output